Purpose of Uml class diagram

→ shows static structure → Structure of the program that doesn't change when the program is running

→ helpful for developers, analyst

myclass

+ attribute 1 : int → d.type
− att 2 : float
att3 : cruc

attributes of class named myclass.
+ → public, − → private
→ protected (only class and subclasses can use)

+ M1 (p1 : bool) : String
− M2 (p6 : int) : class6*

M1 → methods
p1 → para
M2 → returntype of M2.

Class Relationships:

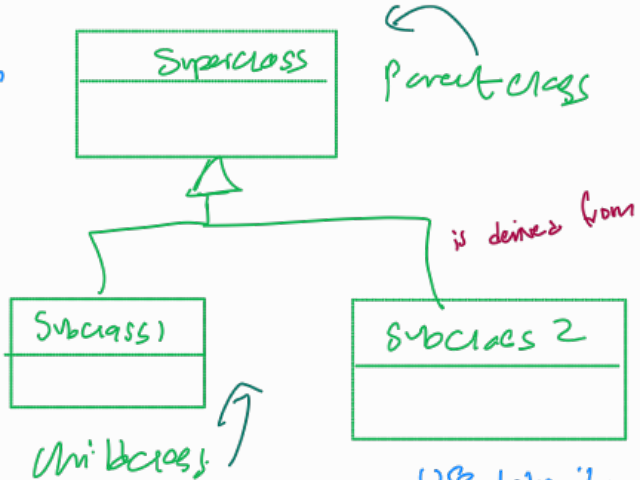
1) Inheritance

Basically child shares everything with parents and child can add more features.

The child class inherits attributes and methods from parent

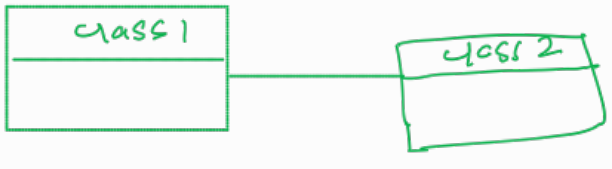
e.g. Student is a person

Person ← Student



Use Inheritance when 1 class naturally extends features to another class

Associations



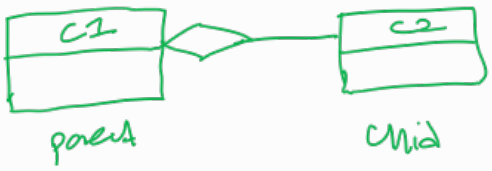
C1 knows about C2 and can call methods of other and vice versa.

Line has no arrows both can talk



Now only C1 can access C2 and not C2 accessing C1. C2 doesnot know about C1

Aggregation:

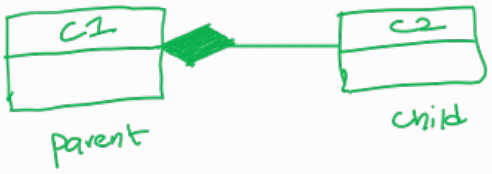


C2 is a part of C1. denoted by *.

C2 can live without C1.!!

eg Books in library

composition :



C2 is a part of C1 and when C1 is destroyed C2 is automatically destroyed

eg A car has engine

Dependency:



C1 depends on C2
C1 uses C2 only for short time

(no ownership is involved)

Inheritence:

Initializer list → initialize data members of class

TO assign data in class we can either use constructor or Initializer list.

↓ more faster as data is assigned during | at the moment the class is created.

```
Class A
{
    int x;
    public
    A() {
        x=10;
    }
};
```

```
class A {
    int x;
public:
    A() : x(10) { } => Initializer
};
```

In some cases we need initializer list as constructors cannot use to assign data

1) constant variable

```
class A {
    const int x;
  public:
    A() : x(10) { }
};
```

✓

```
class A {
    const int x;
  public:
    A() {
        x=10;
    }
};
```

✗

Because const values cannot be changed after the object is created

Syntax of initializer list

```
class name (parameters): member 1 (value 1), member 2 (value 2) {
    constructor's body (optional)
}
```

A() { } → no initializations

```
int a=10;
int &r = a;   // r is another name of a
```

```
class A {
    int &r;
  public: A(int &x) {
        r=x;
    }
};
```

(X)

A ref must be created to a variable

```
class A {
    int &r;
public:
    A(int &x) : r(x) {
    }
};
```

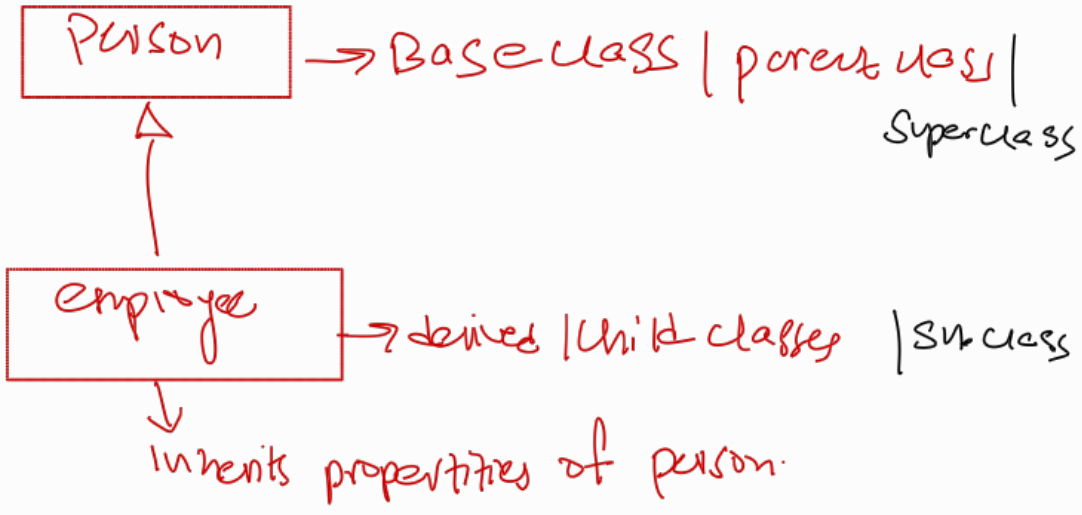
Intializerlist runs first (gives the intial values) then the body of constructor runs for extra logic.

→ object is fully created here

Things that need to be intialized before the body runs:

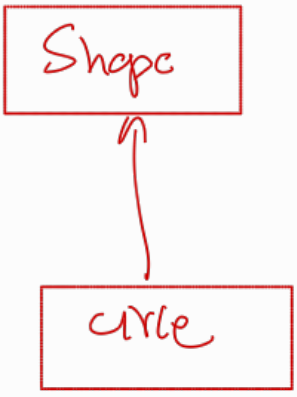
const members
ref members
base class const
member objects that need para

Inheritance

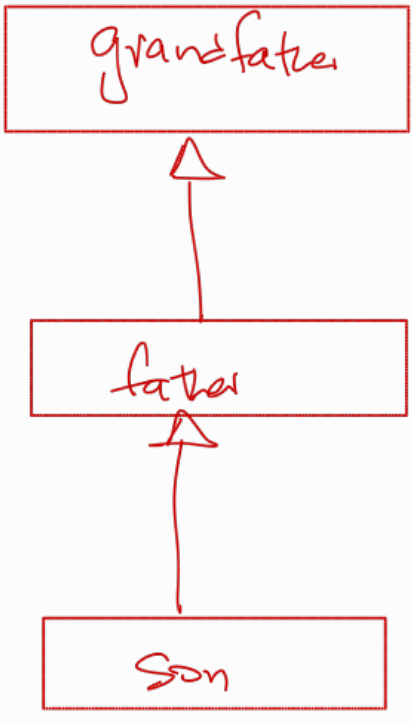


Arrows always point from Subclass to Superclass

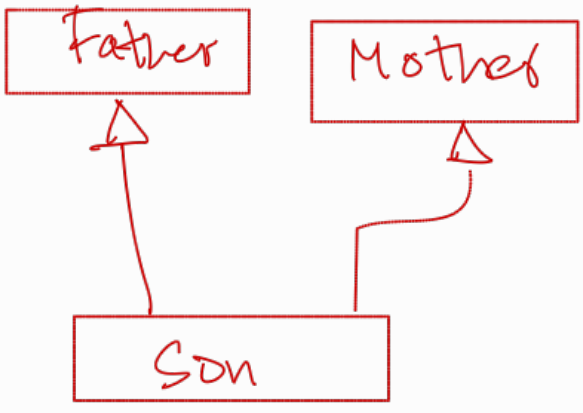
Single Inheritance : A derived class inherits from only 1 Base class



multilevel inheritance:



multipe inheritance: A class inherits from more than 1 base class



Syntax of derived class

Public, private, protected.

class classname : member access Specifier base classname

{

member list;

};



A derived class cannot access private members of Base class. → use getters.

Protected → members that can only be accessed inside the class and derived classes and not outside

(Defining a Subclass from Superclass)

class y : public x {

};

Defining a Subclass from multiple Superclasses

class A : public x, public y, public z {

};

→ multiple inheritance

---

class Super {

public:

void greet() { cout << "hello"; }

};

class Sub : public Super {

// No new variables

};

this does not mean that

Sub class is a copy of Super class. They have diff objects.

if we don't specify any member access variable it is by default private

```
class Super {
    public
        void putij { }
        void getij { }
};

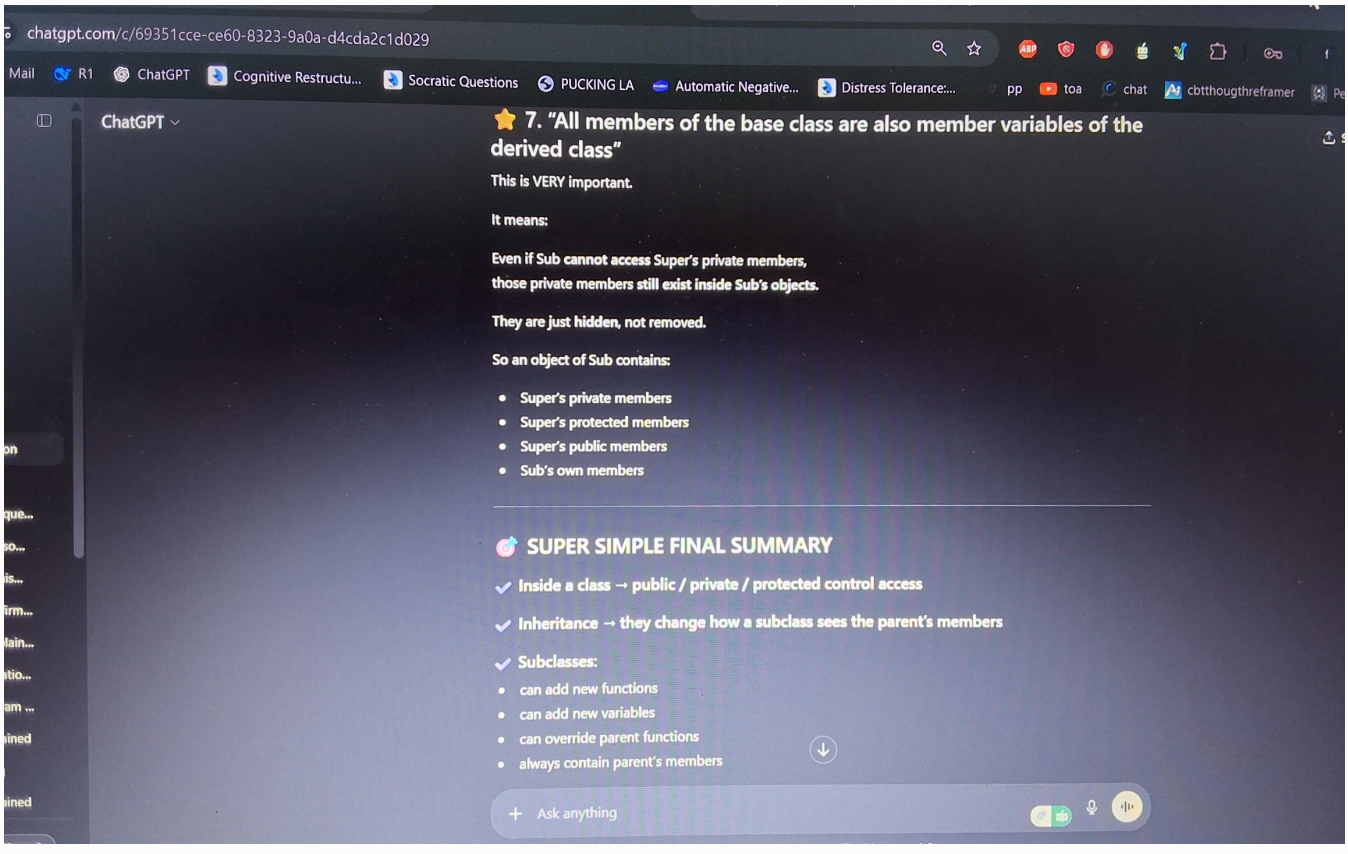
class Sub : Super {
};

Sub S
S.putij;
S.getij; }
```

error

→ Private Inheritance

Because of ↓ the public methods of Super become private inside Sub.

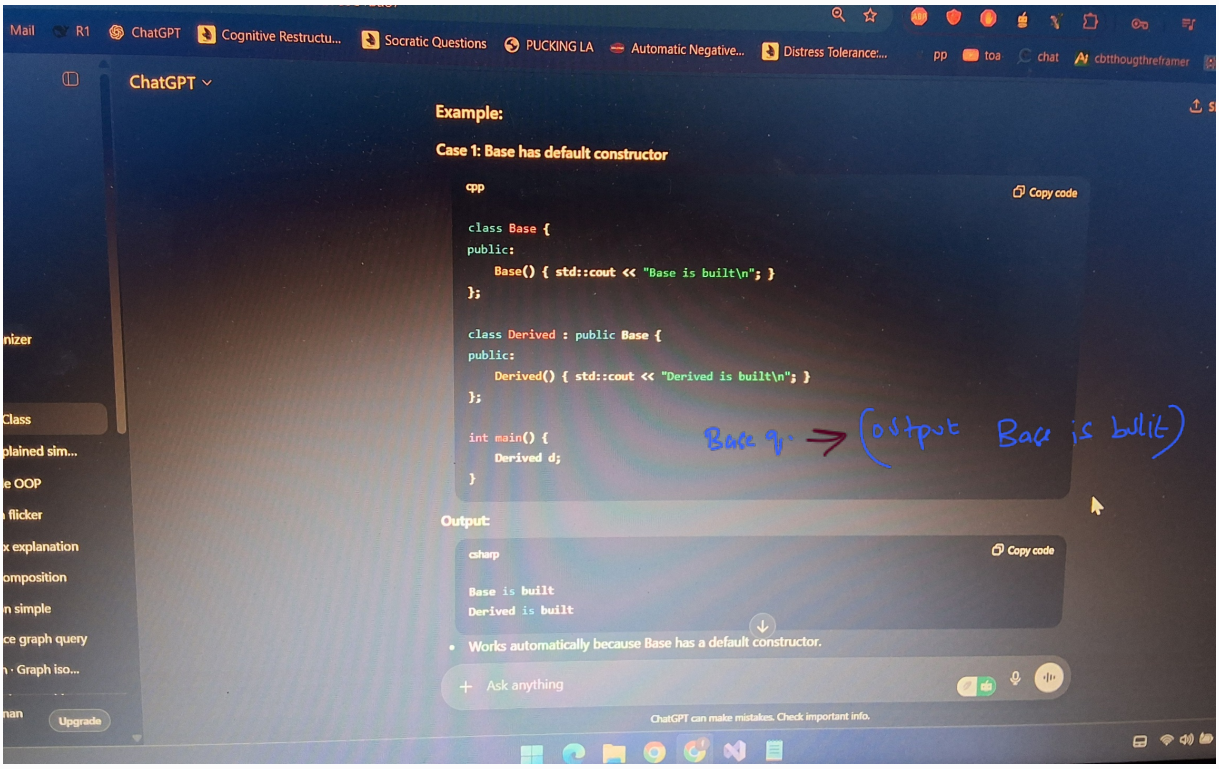



```
class Super {
public:
    void show() {
        cout << "hello";
    }
};

class Sub : public Super {
public:
    void show() {
        cout << "yolo";
    }
};
```

// override

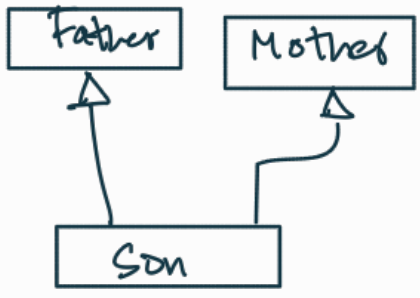
Note:: A derived class is made of derived part + the Base class part



when we create a derived object the Base constructor runs first.

Protected :→ only members b/w classes can be shared.

multiple inheritence syntax



class derived : membermode Base class 1, member mode Base class 2

---

if Base class doesnot have a default constructor::

```
class Base {
public:
    Base(int x) {}
};
class derived : public Base {
public
    derived() {}
};
```

→ So Base cannot be created without giving an integer.

derived d; → Behind cpp does Part 1 Build Base part
Part 2 Build derived part

Now Base needs a int? and derived class will give the Base class that int.

```
class derived : public Base {

public
  derived() : Base(50) {
  }
};
```

→ creates Base part with int.
then create derived class.

Simple

→ if Base class constructor doesn't have any arguments

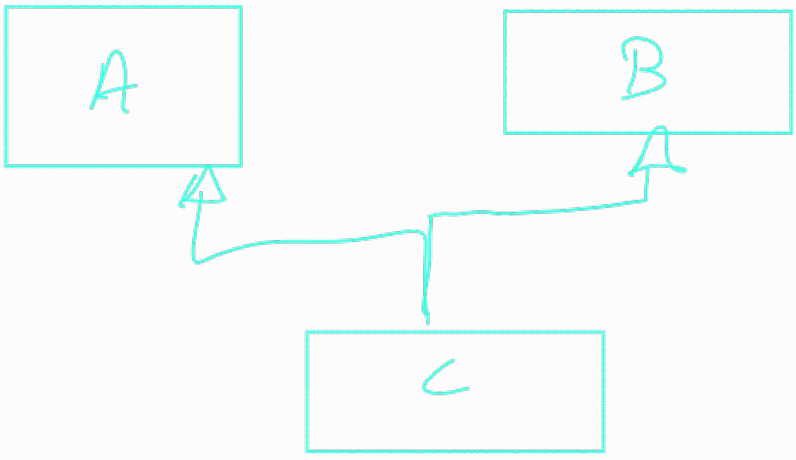
A
↓
B

Obj1 ↙

automatically inherits A constructor

→ But if Base constructor has arguments then the derived class will need to pass arguments to the Base class.

note: Base class constructor is executed first.



if A is declared first then

A constructor execute first.

and vice versa.